The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch26_system_design

AYuSh

Contents

Chapter 26: ML System Design

The ML system design interview asks a different question than everything before this point in the book: not "do you understand this algorithm?" but "can you turn a vague business goal into a working, measurable, scalable ML system — and defend every choice along the way?" It is the round that most cleanly separates senior candidates from junior ones, because the skills it probes — framing, decomposition, metric design, and honest trade-off analysis — are exactly the skills the job consists of. This chapter covers the recurring anatomy behind nearly every design question: framing a business problem as an ML problem (choosing the label, the objective, and the metric hierarchy); the candidate generation → ranking → re-ranking funnel that virtually every large-scale recommendation, search, ads, and feed system shares; embedding-based retrieval at the scale where brute force stops working; the cold-start problem; and the online-versus-offline evaluation gap that decides whether an offline win is real. It closes with walkthroughs of the six canonical design questions — recommendation, search ranking, fraud detection, ad click prediction, feed ranking, and spam filtering — each mapped onto the shared anatomy.

As in the preceding chapters, the claims are measured, not asserted. The listings are from-scratch simulations of the *system* mechanisms: a two-stage funnel reaching recall@10 of 0.37 where the expensive model alone (latency-capped at scoring 1% of the corpus) manages 0.01 and the cheap model alone 0.03, with recall climbing 0.10 → 0.48 as the candidate pool grows from 50 to 5,000 (Listing 1); a from-scratch IVF index answering nearest-neighbor queries 35× faster than brute force by scanning 0.4% of the database at recall 0.22, with the nprobe dial trading up to recall 0.94 at 25% scanned (Listing 2); a CTR model trained on negatives downsampled to 5% whose raw predictions average 0.14 against a true rate of 0.018 (ECE 0.123 → 0.0007 after the closed-form correction), where calibration changes nothing about AUC (0.919 either way) but recovers ~19% of auction value because expected-value ordering mixes the probability with a bid (Listing 3); a matrix-factorization recommender whose cold-item recall@10 is 0.055 — indistinguishable from random (0.050) — while a content-to-latent bridge reaches 0.602, above even the warm-item 0.537 (Listing 4); a naive click model and an inverse-propensity-weighted one that are inseparable by offline AUC on logged clicks (0.909 vs 0.908) yet clearly separated online (true-best-item-at-rank-1: 0.282 vs 0.343), plus the A/B arithmetic showing a +2% relative lift on a 2% CTR needs about a million users per arm for 50% power (Listing 5); a fraud detector at a 0.2% positive rate where ROC-AUC reads a rosy 0.982 while PR-AUC reads the honest 0.425, and where the cost-optimal threshold (miss = $500, review = 5$) cuts expected cost from $59.6k to$ 36.6k versus the default 0.5 (Listing 6); and a feed ranker where a CTR-only objective produces a 100%-clickbait feed with *negative* long-term value, a multi-head value model fixes the objective but still yields a single-topic feed, and MMR re-ranking buys full topic coverage for a 33% value haircut at λ=0.5 (Listing 7).

Framing business problems as ML problems

Every design interview starts the same way, and so should you: **refuse to design until the problem is framed.** A business goal ("increase engagement", "reduce fraud losses", "help users find products") is not an ML problem; the translation has four decisions, each of which the interviewer wants to hear you make explicitly.

First, decide whether ML is warranted at all. ML earns its complexity when the mapping from inputs to decisions is too complex to hand-write, changes over time, and admits a measurable objective with abundant feedback data. If a rule ("block transactions over $10k from new accounts in country X") captures most of the value, say so — proposing a heuristic baseline first is a strength signal, not a weakness, because every production system needs the baseline anyway as a sanity check and a fallback.

Second, choose the prediction target — the label — and be paranoid about it. This is the highest-leverage decision in the entire design because the system will optimize *exactly* what you label, not what you meant. "Engagement" is not a label; a click within this session is. And each concrete choice has failure modes you should name: clicks are abundant and immediate but reward clickbait (Listing 7 measures precisely this); purchases are closer to value but sparse and delayed; explicit ratings are gold but scarce and biased toward strong opinions; "reported as fraud" arrives weeks late and only for transactions you *allowed* (label censorship — the ones you blocked never get a label). Almost every real system uses **implicit feedback** (clicks, watches, dwell time) because of its volume, and therefore inherits its biases — position bias above all, which Listing 5 quantifies.

Third, translate the goal into a metric hierarchy, because no single number serves all masters: a **business/north-star metric** (revenue, retention, fraud losses) that moves slowly and can only be read in an A/B test; one or two **online product metrics** (CTR, session length, report rate) that the A/B test actually measures; and **offline model metrics** (AUC, recall@k, NDCG, calibration error) that you can compute before shipping anything. The interviewer wants you to state the hierarchy *and* the ways it can come apart — offline AUC up while online CTR is flat (Listing 5's disagreement), or CTR up while long-term retention falls (Listing 7's clickbait trap). Metrics also need **guardrails**: a feed ranker optimized for engagement should be shipped with hide/report rates and content-diversity counters watched in the same experiment.

Fourth, state the constraints, because they select the architecture. The numbers that matter: corpus size (millions to billions of items), request rate (thousands to millions of QPS), latency budget (tens of milliseconds for the whole page, of which ranking gets a slice), and freshness (how fast must a new item, a new user action, or a new fraud pattern show up in predictions?). One sentence of arithmetic justifies the entire funnel architecture of the next section: if you have 100 ms and a ranker that costs 1 ms per item, you can rank a few hundred items — and the corpus has a hundred million. Everything else follows.

The serving funnel: candidate generation → ranking → re-ranking

Every large-scale recommendation, search, ads, and feed system converges on the same shape, because the same arithmetic forces it: a **funnel** of successively more expensive models over successively fewer items.

Stage 1 — candidate generation (retrieval) reduces the corpus (10^6–10^9 items) to hundreds or a few thousand candidates in a few milliseconds. The models here must be cheap *per item at scale*, which in practice means embedding dot products served by an approximate nearest neighbor index (next section), plus non-ML sources run in parallel: items from followed authors, trending/popular items, co-occurrence lists, business rules. Multiple candidate sources are the norm — each covers a different recall failure of the others, and their union is deduplicated before ranking. Retrieval is optimized for **recall**: its only job is to not lose the good items; precision is the next stage's job.

Stage 2 — ranking scores the surviving hundreds with the expensive model: a deep network with rich **cross-features** between the user and each item (user's historical CTR on this category, item's CTR in this context) that retrieval structurally cannot use, because retrieval's cheapness depends on the user and item sides being separable (a dot product) so item vectors can be pre-indexed. This stage is optimized for **precision at the top** — NDCG, AUC, calibrated probabilities.

Stage 3 — re-ranking applies list-level logic the pointwise ranker cannot see: diversity (Listing 7's MMR), freshness boosts, deduplication of near-identical items, business rules and policy filters, exploration slots. It reorders dozens; it can afford anything.

Listing 1 measures why the funnel is not merely conventional but *dominant*. In a 100,000-item corpus where a latency budget allows the expensive ranker to score only 1,000 items: the expensive model alone on a random affordable subset gets recall@10 of 0.01 (it never sees the good items); the cheap model alone over everything gets 0.03 (it sees everything but cannot rank); the funnel — cheap model retrieves 1,000, expensive model ranks those — gets 0.37. Each stage supplies what the other lacks: coverage from the cheap stage, discrimination from the expensive one. The listing also sweeps the candidate-pool size: recall@10 climbs 0.10 → 0.20 → 0.36 → 0.48 as the pool grows 50 → 5,000, the diminishing-returns curve that in production sets the pool size as a latency/quality dial. Interview synthesis: *the funnel exists because retrieval errors are unrecoverable* — an item the first stage drops is invisible to every later stage — *so you buy recall cheaply at the top and spend your latency budget on precision at the bottom.*

Embedding-based retrieval at scale

The workhorse of candidate generation is the **two-tower model**: a user tower encodes the user (profile, history) into a vector, an item tower encodes each item into a vector of

the same dimension, trained (typically with in-batch sampled-softmax/contrastive loss over positive interactions) so that relevance ≈ dot product. The separability is the entire point: item vectors depend only on items, so all N item vectors are computed offline and stored in an index; at request time you compute one user vector and ask the index for its top-k neighbors. What would be N forward passes becomes one forward pass and one nearest-neighbor query.

But *exact* nearest-neighbor search is O(N·d) per query — Listing 2 measures 3.7 ms at N=200k, d=64, which extrapolated to 10^8 items is seconds, hopeless at serving QPS. **Approximate nearest neighbor (ANN)** indexes trade a controlled amount of recall for orders of magnitude of speed. The listing builds the simplest important one from scratch, **IVF (inverted file)**: k-means the corpus into C cells offline; at query time, score the query against the C centroids, and exhaustively search only the closest `nprobe` cells. The measured dial: nprobe=1 scans 0.4% of the database for recall@10 of 0.22 at 0.1 ms; nprobe=4 → 0.46 at 1.6% scanned; nprobe=16 → 0.76 at 6.3%; nprobe=64 → 0.94 at 25% scanned and 4.1 ms — a smooth recall-versus-cost curve, tuned per application. The other two names to know: **HNSW** (hierarchical navigable small-world graphs — greedy search through a layered proximity graph; higher memory, excellent recall/latency, the default in most vector databases) and **product quantization (PQ)** — compressing vectors into codebook indices so distance computation happens over compressed codes, cutting memory ~10–50× and often composed with IVF (IVF-PQ) for billion-scale corpora. The system-level caveats worth volunteering: the index is built offline, so new items need an insertion path (HNSW supports incremental inserts; IVF assigns to existing cells) and retraining the towers changes the geometry, forcing a full re-index — which is why tower retraining and index rebuilds are scheduled together.

The cold-start problem

Any system whose representations are learned *from interactions* fails on entities that have none: new items, new users, new ads. This is not a soft degradation — Listing 4 measures it as total failure. A matrix-factorization recommender trained on interactions involving 800 warm items achieves recall@10 of 0.537 on warm items, but on the 200 cold items its recall is 0.055 — statistically indistinguishable from random recommendation (0.050) — because a cold item's embedding never received a single gradient update; it is still at its random initialization.

The fixes, in the order you should present them: **content-based bridges** — learn a map from item *features* (text, image, category, price, creator) into the interaction-embedding space using warm items as supervision, then embed cold items through the map. The listing's ridge regression from features to learned MF embeddings lifts cold-item recall to 0.602 — in this synthetic setting even above warm recall, because the features are clean while the interaction signal is noisy; in reality content lands between random and warm, and the ordering (content ≫ random, warm ≥ content) is the robust takeaway. **Popularity and heuristic priors** are the zero-information fallback for brand-new users

(onboarding often shortcuts this by *asking* — pick five topics you like). **Exploration** — giving cold items a small share of deliberately non-greedy impressions (an ε-greedy slice or a bandit with an optimism/uncertainty bonus) — is how cold items *earn* interactions; without it, a system that only shows what already performs well never gathers the data to learn about new items, a feedback loop that entrenches incumbents. And **hybrid architectures** avoid the cliff altogether: if the item tower of a two-tower model consumes content features (not just a learned ID embedding), every new item gets a usable vector at upload time, and the ID embedding — which captures what content cannot — is learned on top as interactions accumulate. The user-side story is symmetric: new users get population priors, onboarding signals, and contextual features (device, geo, time) until they have history.

Online vs offline evaluation

Offline evaluation — computing metrics on held-out logged data — is fast, free, and safe, and it is where every model iteration starts. It is also systematically untrustworthy for ranking systems, for a reason worth stating precisely: **the logs were generated by the old system.** You only observe clicks on items the old ranker chose to show, at the positions it showed them, and users click for reasons entangled with those positions. Two consequences follow. **Position bias**: an item shown at rank 1 gets clicked more than the same item at rank 10, so click-trained models learn "was shown high by the old ranker" as if it were relevance — self-reinforcing, since the model is then used to decide what to show high. **Off-policy mismatch**: your new model's preferred ranking was never shown, so its consequences are simply absent from the data.

Listing 5 builds the trap and measures it. Clicks are simulated as relevance × exposure, with exposure decaying in the position assigned by an old ranker only weakly correlated with true relevance. A naive model trained on raw clicks and an **inverse propensity scoring (IPS)** model — identical except each training example is weighted by 1/P(exposure), unbiasing the click signal — are then scored both ways. Offline, on held-out *logged clicks*, they are indistinguishable: AUC 0.909 vs 0.908, and the naive model even edges ahead, because predicting logged clicks rewards imitating the old ranker's biases. Online, against *true relevance*, the IPS model is clearly better: NDCG 0.920 vs 0.938, and it places the truly-best item at rank 1 in 34.3% of queries versus 28.2%. The offline metric on biased logs *cannot see* the improvement — which is why counterfactual evaluation (IPS and its variants), click models that estimate propensities, and randomization injected into logging (small shuffles among top positions) are standard tools, and why no ranking change ships on offline numbers alone.

The gold standard is the **A/B test**: randomize users into control and treatment, serve each arm its own system, compare online metrics with a significance test. Its cost is time and traffic, and the arithmetic is unforgiving — Listing 5's power calculation shows that detecting a +2% *relative* lift on a 2% CTR (i.e., 2.00% → 2.04%) with 80%+ power needs several million users per arm; at 10^5 per arm the power is 0.09, meaning the test almost

always misses a real win. This is why experimentation platforms invest in variance reduction (CUPED, stratification) and why **interleaving** — merging the two rankers' outputs into one list and attributing clicks per ranker — is used for ranking comparisons: it is typically 10–100× more sensitive per impression, at the cost of only answering "which ranker is preferred" rather than measuring business-metric deltas. Standard A/B hygiene worth naming: an A/A test to validate the pipeline, guardrail metrics alongside the target metric, novelty/primacy effects (run long enough to wash out), and pre-registered sample sizes rather than peeking.

Canonical design walkthroughs

Six systems cover most of what is asked. Each is the shared anatomy — framing, funnel, features, evaluation — with different stresses. What follows is the skeleton you adapt, not a script you recite; in the interview, spend your time where the system's characteristic difficulty lies.

Recommendation system (e.g., movies, products). Label: implicit engagement (watch ≥ N minutes, purchase), not ratings — volume wins. Funnel: multiple candidate sources (two-tower ANN, co-occurrence "users who watched X", trending, followed creators) → DNN ranker with user-history cross-features → re-rank for diversity and freshness. Characteristic stresses: cold start on both sides (this is where Listing 4's content bridge and exploration slots belong) and feedback loops — the system trains on data it created, so popular items accumulate advantage; counter with propensity weighting and exploration. Metrics: offline recall@k for retrieval, NDCG for ranking; online engagement with retention as north star and diversity as guardrail.

Search ranking. The new element is the **query**: understanding (spell correction, expansion, intent), and retrieval that must blend **lexical** matching (BM25/inverted index — exact terms, model numbers, rare entities) with **semantic** matching (two-tower dense retrieval — synonyms, paraphrase), because each fails where the other succeeds; hybrid retrieval with score fusion is the standard answer. Ranking features: text-match scores, document quality/authority, engagement history, recency for newsy queries. Labels: human relevance judgments (expensive, clean) plus click data (abundant, position-biased — Listing 5 applies with full force; this is where you say "IPS" out loud). Metrics: NDCG@10 offline, interleaving online.

Fraud detection. The stresses: extreme class imbalance, adversarial drift, asymmetric costs, and delayed/censored labels (chargebacks arrive weeks late; blocked transactions never get labels). Listing 6 carries the metric argument — at a 0.2% fraud rate, accuracy is vacuous (99.8% for "always legit"), ROC-AUC flatters (0.982) because it credits ranking among the overwhelming negatives, and PR-AUC tells the truth (0.425). The decision layer is cost-sensitive, not symmetric: with a missed fraud at $500 and a manual review at $5, the cost-minimizing threshold flags 2.8% of transactions and catches 88% of fraud at $36.6k expected cost, versus $59.6k at the default 0.5 threshold — the threshold is a

business decision computed from the cost matrix, and typically three-way (approve / review queue / block). Architecture notes: real-time scoring within the payment authorization budget (tens of ms) forces precomputed entity aggregates (velocity features: transactions per card per hour, distinct devices per account per day) served from a feature store; rules run beside the model (interpretable, instant to deploy against a new attack); the model retrains frequently because fraudsters adapt to it — the truest adversarial drift in mainstream ML.

Ad click prediction (CTR). The unique requirement, and the reason interviewers love this question: ads need the **probability itself**, not just the ordering, because the auction ranks by expected value = pCTR × bid and pricing depends on it. Listing 3 is the whole argument in miniature. Training on negatives downsampled to 5% (standard at ad scale, where positives are ~1%) inflates raw predicted CTR to 0.141 against a true 0.018; the closed-form correction q = p/(p + (1−p)/s) restores calibration (ECE 0.123 → 0.0007). AUC is untouched (0.919 both ways — the correction is monotone), so a ranking-only system would never notice; but ranking by *miscalibrated* pCTR × bid realizes 919 units of value where the calibrated ranking realizes 1,095 (oracle: 1,097) — miscalibration mixes with heterogeneous bids to reorder the auction and burn ~19% of the value. Say "downsampling correction" and "calibration is a launch-blocking metric for ads" and this question is largely won. Remaining stresses: massive sparse categorical features (hashed embeddings), feature crosses (the FM/DCN lineage), freshness (online/continual training against ad churn), and delayed conversion feedback if optimizing beyond the click.

Feed ranking. The characteristic difficulty is the **objective**: engagement is many actions with different meanings. Listing 7's progression is the answer's spine. Rank by pCTR alone: the feed is 100% clickbait topic, hide-probability mass 0.37, and *negative* total long-term value. Rank by a **multi-head value model** — separate heads predicting click, like, share, hide, each cheap to retarget, combined as w·[pclick, plike, pshare, phide] with business-tuned weights (the listing's 1·click + 3·like + 8·share − 20·hide) — and value recovers (7.29) with hides collapsing to 0.04; the weights, not the heads, encode the product's values, and they are set by leadership + experimentation, which is exactly what you should say. But the value-ranked feed is still 100% one topic; **MMR re-ranking** (pick argmax λ·value − (1−λ)·max-similarity-to-picked) trades value for coverage smoothly: λ=0.9 keeps 99.6% of value and cracks the monoculture; λ=0.5 covers all 8 topics (top-topic share 35%) for a 33% value haircut. Add the temporal notes — sessions need freshness, heavy-hitter authors need fatigue penalties — and the guardrail note: hide/report rates and diversity ship in the same A/B as engagement.

Spam filter. Superficially binary classification; the actual stresses are asymmetric costs *in the opposite direction from fraud* (a false positive — real mail in the spam folder — is the catastrophic error, so the operating point pins the false-positive rate very low, e.g. thresholds set for FPR ≤ 0.1%, and the metric is recall at that pinned FPR), adversarial adaptation (spammers probe the filter; features and models must refresh continuously), and signals beyond content: sender reputation (domain/IP history, SPF/DKIM), sending-graph features (a burst of identical mails to unrelated recipients), and user feedback

("mark as spam" / "not spam") as a continuous label stream. A useful closing note in the interview: personalization (one user's spam is another's newsletter) and the layered defense — IP blocklists and rate limits before the ML model ever runs, the same rules-beside-model architecture as fraud.

Code implementations

(Each listing is a self-contained from-scratch simulation of one system mechanism over controlled synthetic data, so the architectural claim — not a particular dataset or model — is what is measured. The numbers quoted in the prose are these programs' actual output.)

Listing 1 — The funnel: why every big system is multi-stage

A 100,000-item corpus, a cheap noisy scorer, an expensive accurate scorer that latency caps at 1,000 items. Neither alone works; the funnel does — and the candidate-pool size is a recall dial.

```
"""Listing 1: why ranking is a funnel. Corpus of N items with a true relevance score
per query.
Two scorers: a CHEAP retrieval model (true score + large noise, ~free per item) and an
EXPENSIVE
ranker (true score + small noise, but latency allows scoring only B items per query).
Strategies:
(a) expensive on a random B items; (b) cheap only over the full corpus; (c) two-stage
funnel:
cheap retrieves top-M, expensive re-scores those M (M<=B), serve top-k. Metric:
recall@10 of the
true top-10. The funnel dominates because each stage does what it is good at."""
import numpy as np
rng=np.random.default_rng(0)
N=100_000; k=10; B=1_000; Q=200
def recall_at_k(order, true_top):
    return len(set(order[:k]) & set(true_top))/k
res={"expensive_random_B":[], "cheap_full":[], "funnel":[]}
for q in range(Q):
    true=rng.normal(0,1,N)
    cheap=true+rng.normal(0,1.5,N)          # fast, noisy (e.g. two-tower dot product)
    true_top=np.argpartition(-true,k)[:k]
    # (a) expensive model, but latency only allows a random subset of B
    sub=rng.choice(N,B,replace=False)
    exp_sub=true[sub]+rng.normal(0,0.3,B)
    res["expensive_random_B"].append(recall_at_k(sub[np.argsort(-exp_sub)],true_top))
    # (b) cheap model over everything
    res["cheap_full"].append(recall_at_k(np.argsort(-cheap)[:k],true_top))
    # (c) funnel: cheap retrieves M=B candidates, expensive re-ranks them
    cand=np.argpartition(-cheap,B)[:B]
    exp_c=true[cand]+rng.normal(0,0.3,B)
    res["funnel"].append(recall_at_k(cand[np.argsort(-exp_c)],true_top))
for kk,v in res.items(): print(f"{kk:22s} recall@10 = {np.mean(v):.2f}")
# sweep candidate pool size M for the funnel
for M in [50,200,1000,5000]:
    r=[]
    for q in range(100):
        true=rng.normal(0,1,N); cheap=true+rng.normal(0,1.5,N)
        true_top=np.argpartition(-true,k)[:k]
        cand=np.argpartition(-cheap,M)[:M]
        exp_c=true[cand]+rng.normal(0,0.3,M)
```

```
        r.append(recall_at_k(cand[np.argsort(-exp_c)],true_top))
    print(f"funnel M={M:5d}: recall@10 = {np.mean(r):.2f}")
```

Listing 2 — Embedding retrieval at scale: an IVF index from scratch

Brute-force nearest neighbor versus a k-means inverted-file index. The `nprobe` parameter is the recall/latency dial every ANN system exposes.

```
"""Listing 2: embedding-based retrieval at scale — a from-scratch IVF index. Brute-force nearest
neighbor over N vectors costs O(N*d) per query; an inverted-file (IVF) index k-means-clusters the
corpus into C cells and scans only the nprobe closest cells. We measure recall@10 vs the fraction
of the database scanned — the accuracy/cost dial every ANN system exposes."""
import numpy as np, time
rng=np.random.default_rng(0)
N,d,C=200_000,64,256
X=rng.normal(0,1,(N,d)).astype(np.float32)
# make clustered structure (realistic embeddings are clustered)
centers=rng.normal(0,0.7,(C,d)).astype(np.float32)
assign=rng.integers(0,C,N); X+=centers[assign]
X/=np.linalg.norm(X,axis=1,keepdims=True)
# train coarse quantizer: mini k-means (few iters is fine)
cent=X[rng.choice(N,C,replace=False)].copy()
for _ in range(5):
    a=np.argmax(X@cent.T,axis=1)   # cosine assignment
    for c in range(C):
        m=X[a==c]
        if len(m): cent[c]=m.mean(0)/np.linalg.norm(m.mean(0))
a=np.argmax(X@cent.T,axis=1)
lists=[np.where(a==c)[0] for c in range(C)]
Qn=100; Qs=X[rng.choice(N,Qn)]+rng.normal(0,0.4,(Qn,d)).astype(np.float32)
Qs/=np.linalg.norm(Qs,axis=1,keepdims=True)
t0=time.time(); truth=[np.argpartition(-(X@q),10)[:10] for q in Qs]; bf=(time.time()-t0)/Qn*1000
print(f"brute force: {bf:.1f} ms/query (N={N}, d={d})")
for nprobe in [1,4,16,64]:
    rec=0; scanned=0; t0=time.time()
    for qi,q in enumerate(Qs):
        cells=np.argpartition(-(cent@q),nprobe)[:nprobe]
        ids=np.concatenate([lists[c] for c in cells]); scanned+=len(ids)
        top=ids[np.argpartition(-(X[ids]@q),min(10,len(ids)-1))[:10]]
        rec+=len(set(top)&set(truth[qi]))/10
    ms=(time.time()-t0)/Qn*1000
    print(f"IVF nprobe={nprobe:3d}: recall@10={rec/Qn:.2f}  scanned={scanned/Qn/N*100:4.1f}% of DB  {ms:.1f} ms/query")
```

Listing 3 — CTR prediction: downsampling, calibration, and auction value

Training on downsampled negatives inflates probabilities; the closed-form correction restores calibration without touching AUC — and calibration, not ranking, is what the ad auction consumes.

```
"""Listing 3: ad CTR prediction — imbalance, downsampling, and why calibration matters. True CTRs
average ~1%. Training on all data is dominated by negatives, so production systems downsample
```

```
negatives by rate s — but that INFLATES predicted probabilities, and ads systems
consume the raw
probability (expected value = pCTR * bid), not just the ranking. The closed-form fix:
q = p / (p + (1-p)/s). We measure calibration before/after and the revenue ranking
impact."""
import numpy as np
from sklearn.linear_model import LogisticRegression
rng=np.random.default_rng(0)
n,dt=400_000,20
X=rng.normal(0,1,(n,dt))
w=rng.normal(0,0.5,dt); logit=X@w-6.0           # base rate ~1%
p_true=1/(1+np.exp(-logit)); y=(rng.random(n)<p_true).astype(int)
print(f"positive rate: {y.mean():.4f}")
Xtr,ytr,Xte,yte,pte=X[:300_000],y[:300_000],X[300_000:],y[300_000:],p_true[300_000:]
s=0.05  # keep 5% of negatives
keep=(ytr==1)|(rng.random(300_000)<s)
m=LogisticRegression(max_iter=1000).fit(Xtr[keep],ytr[keep])
p=m.predict_proba(Xte)[:,1]
q=p/(p+(1-p)/s)                                  # downsampling correction
def ece(pred,lab,bins=10):
    e=0; edges=np.quantile(pred,np.linspace(0,1,bins+1))
    for i in range(bins):
        m_=(pred>=edges[i])&(pred<=edges[i+1])
        if m_.sum(): e+=m_.mean()*abs(pred[m_].mean()-lab[m_].mean())
    return e
print(f"mean predicted CTR raw: {p.mean():.4f}   corrected: {q.mean():.4f}   actual:
{yte.mean():.4f}")
print(f"ECE raw: {ece(p,yte):.4f}   ECE corrected: {ece(q,yte):.4f}")
# ranking (AUC) is unchanged by the monotone correction — but expected-value ORDERING
with bids is not
def auc(sc,lab):
    r=sc.argsort().argsort(); pos=lab==1
    return (r[pos].mean()-r[~pos].mean())/len(lab)+0.5
print(f"AUC raw: {auc(p,yte):.3f}   AUC corrected: {auc(q,yte):.3f}  (identical:
monotone)")
bids=rng.lognormal(0,1,len(p))                   # per-ad bids
ev_raw, ev_cal, ev_true = p*bids, q*bids, pte*bids
pick_raw=np.argsort(-ev_raw)[:1000]; pick_cal=np.argsort(-ev_cal)[:1000];
best=np.argsort(-ev_true)[:1000]
print(f"realized value, raw ranking: {ev_true[pick_raw].sum():.0f}  calibrated:
{ev_true[pick_cal].sum():.0f}  oracle: {ev_true[best].sum():.0f}")
```

Listing 4 — Cold start, measured

Matrix factorization is exactly random on items with no interactions; a content-to-latent bridge recovers them.

```
"""Listing 4: the cold-start problem, measured. Users and items have true latent
factors; observed
interactions exist only for 'warm' items. Matrix factorization (SGD on observed pairs)
learns great
embeddings for warm items but its cold-item embeddings never receive a gradient —
recall for cold
items is ~random. A content model (a linear map from item features to the latent space,
trained on
warm items) transfers to cold items, and popularity is the zero-information
fallback."""
import numpy as np
rng=np.random.default_rng(0)
nu,ni,dl,dfeat=2000,1000,8,32
U=rng.normal(0,1,(nu,dl)); V=rng.normal(0,1,(ni,dl))
F=V@rng.normal(0,1,(dl,dfeat))+rng.normal(0,0.5,(ni,dfeat))  # item features correlate
w/ latents
cold=np.arange(ni)>=800                                       # last 200 items: no
```

```
interactions
# observed interactions: each user interacts with their top warm items (+noise)
scores=U@V.T+rng.normal(0,1.0,(nu,ni)); scores[:,cold]=-np.inf
obs=[np.argpartition(-scores[u],20)[:20] for u in range(nu)]
# --- matrix factorization on observed pairs (BPR-flavored SGD) ---
P=rng.normal(0,.1,(nu,dl)); Q=rng.normal(0,.1,(ni,dl)); lr=0.05
warm_ids=np.where(~cold)[0]
for ep in range(60):
    for u in range(nu):
        for i in rng.choice(obs[u],8):
            j=rng.choice(warm_ids)                              # negative sample
            x=P[u]@(Q[i]-Q[j]); g=1/(1+np.exp(x))               # BPR gradient
            P[u]+=lr*g*(Q[i]-Q[j]); Q[i]+=lr*g*P[u]; Q[j]-=lr*g*P[u]
# --- content model: ridge regression F -> Q on warm items, predict cold embeddings ---
A=np.linalg.solve(F[~cold].T@F[~cold]+10*np.eye(dfeat), F[~cold].T@Q[~cold])
Qc=Q.copy(); Qc[cold]=F[cold]@A
# --- evaluate: for each user, true top-10 among COLD items only ---
true_cold=U@V[cold].T
def recall(embU,embI):
    r=0
    for u in range(500):
        pred=embU[u]@embI[cold].T
        t=set(np.argpartition(-true_cold[u],10)[:10])
        r+=len(t&set(np.argpartition(-pred,10)[:10]))/10
    return r/500
pop=np.zeros(ni)  # popularity: all-zero for cold items -> random among cold
print(f"cold-item recall@10, MF embeddings:       {recall(P,Q):.3f}")
print(f"cold-item recall@10, content->latent map: {recall(P,Qc):.3f}")
print(f"cold-item recall@10, random/popularity:   {10/cold.sum():.3f}")
# warm-item sanity check
true_warm=U@V[~cold].T
r=0
for u in range(500):
    pred=P[u]@Q[~cold].T
    t=set(np.argpartition(-true_warm[u],10)[:10])
    r+=len(t&set(np.argpartition(-pred,10)[:10]))/10
print(f"warm-item recall@10, MF embeddings:       {r/500:.3f}")
```

Listing 5 — Offline metrics on biased logs vs online truth, and A/B power

Position-biased clicks make the naive and IPS models indistinguishable offline while the online metric separates them cleanly — plus the sample-size arithmetic of detecting small CTR lifts.

```
"""Listing 5: why offline and online evaluation disagree. Clicks are logged under an
OLD ranker
with position bias: P(click) = relevance * exposure(position). A new model trained/
evaluated on
these logs looks good offline exactly when it imitates the old ranker (it predicts the
clicks the
old ranker generated), not when it ranks by true relevance. We train a naive click
model and an
inverse-propensity-weighted (IPS) one, score both offline (AUC on held-out logged
clicks) and
'online' (NDCG against true relevance) — and the offline metric picks the WRONG
model."""
import numpy as np
rng=np.random.default_rng(0)
nq,ni,dt=12000,20,12
Xf=rng.normal(0,1,(nq,ni,dt))
w_rel=rng.normal(0,1,dt); rel=1/(1+np.exp(-(Xf@w_rel)))       # true relevance
w_old=w_rel*0.2+rng.normal(0,1,dt)                          # old ranker: weakly
```

```
correlated
old_rank=np.argsort(np.argsort(-(Xf@w_old),axis=1),axis=1)    # position of each item
under old ranker
prop=1/(1+old_rank)**1.5                                          # exposure ~ 1/
(1+pos)
click=(rng.random((nq,ni))<rel*prop).astype(float)
X2=Xf.reshape(-1,dt); c2=click.ravel(); pr2=prop.ravel()
tr=slice(0,200_000); te=slice(200_000,None)
def fit_logreg(X,y,sw,iters=800,lr=0.5):
    w=np.zeros(X.shape[1]); b=0.
    for _ in range(iters):
        p=1/(1+np.exp(-(X@w+b))); g=(p-y)*sw
        w-=lr*(X.T@g)/sw.sum(); b-=lr*g.sum()/sw.sum()
    return w,b
w_n,b_n=fit_logreg(X2[tr],c2[tr],np.ones(200_000))             # naive: clicks as
labels
w_i,b_i=fit_logreg(X2[tr],c2[tr],1/pr2[tr])   # IPS: weight clicks by 1/propensity
def auc(sc,lab):
    r=sc.argsort().argsort(); pos=lab==1
    return (r[pos].mean()-r[~pos].mean())/len(lab)+0.5
def ndcg_true(w,b):                                               # 'online': rank by
model, gain = TRUE relevance
    s=0
    for q in range(10000,12000):
        order=np.argsort(-(Xf[q]@w+b))
        dcg=(rel[q][order]/np.log2(np.arange(ni)+2)).sum()
        ideal=(np.sort(rel[q])[::-1]/np.log2(np.arange(ni)+2)).sum()
        s+=dcg/ideal
    return s/2000
print(f"OFFLINE  AUC on logged clicks:  naive={auc(X2[te]@w_n+b_n,c2[te]==1):.3f}
IPS={auc(X2[te]@w_i+b_i,c2[te]==1):.3f}")
def top1_hit(w,b):
# is the truly-best item ranked first?
    h=0
    for q in range(10000,12000):
        h+=(np.argmax(Xf[q]@w+b)==np.argmax(rel[q]))
    return h/2000
print(f"ONLINE   NDCG vs true relevance: naive={ndcg_true(w_n,b_n):.3f}
IPS={ndcg_true(w_i,b_i):.3f}")
print(f"ONLINE   true-best item at rank 1: naive={top1_hit(w_n,b_n):.3f}
IPS={top1_hit(w_i,b_i):.3f}")
# (B) the online side: how big must an A/B test be? Power for a relative CTR lift.
from math import sqrt, erf
def power(p0, rel_lift, n, alpha=0.05):
    p1=p0*(1+rel_lift); se=sqrt(p0*(1-p0)/n+p1*(1-p1)/n)
    z=(p1-p0)/se - 1.96
    return 0.5*(1+erf(z/sqrt(2)))
for n in [10_000,100_000,1_000_000,10_000_000]:
    print(f"n={n:>10,}/arm: power to detect +2% rel lift on 2% CTR = {power(0.02,
0.02,n):.2f}")
```

Listing 6 — Fraud: imbalance, PR vs ROC, and the cost-optimal threshold

At a 0.2% positive rate, accuracy and ROC-AUC flatter; precision-recall and a cost matrix make the real decisions.

```
"""Listing 6: fraud detection — extreme imbalance and cost-sensitive thresholds. At a
0.2% fraud
rate, accuracy is a useless metric (predict-all-legit scores 99.8%) and ROC-AUC is
misleadingly
rosy because it credits ranking among the vast negatives; precision-recall exposes the
truth.
```

```
The operating threshold is a business decision: minimize expected cost given the price
of a missed
fraud vs the price of a false alarm (manual review)."""
import numpy as np
from sklearn.linear_model import LogisticRegression
rng=np.random.default_rng(0)
n,dt=500_000,15; frate=0.002
y=(rng.random(n)<frate).astype(int)
X=rng.normal(0,1,(n,dt)); X[y==1]+=rng.normal(1.2,0.3,(y.sum(),dt))*rng.random(dt)  #
fraud shifts some features
Xtr,ytr,Xte,yte=X[:350_000],y[:350_000],X[350_000:],y[350_000:]
m=LogisticRegression(max_iter=1000,class_weight="balanced").fit(Xtr,ytr)
p=m.predict_proba(Xte)[:,1]
print(f"fraud rate: {yte.mean():.4f}   accuracy of 'always legit': {1-yte.mean():.4f}")
def auc(sc,lab):
    r=sc.argsort().argsort(); pos=lab==1
    return (r[pos].mean()-r[~pos].mean())/len(lab)+0.5
print(f"ROC-AUC: {auc(p,yte):.3f}")
# precision-recall at thresholds + average precision
order=np.argsort(-p); ys=yte[order]
tp=np.cumsum(ys); prec=tp/np.arange(1,len(ys)+1); rec=tp/ys.sum()
ap=np.sum(prec*ys)/ys.sum()
print(f"PR-AUC (average precision): {ap:.3f}   <- the honest number")
for r_ in [0.5,0.8,0.9]:
    i=np.searchsorted(rec,r_)
    print(f"  at recall {r_:.0%}: precision = {prec[i]:.2f} (flag {i+1} txns to catch
{int(tp[i])} frauds)")
# cost-based threshold: missed fraud costs $500, manual review costs $5
C_fn,C_fp=500,5
ths=np.quantile(p,np.linspace(0.5,0.9999,200))
costs=[(C_fn*((p<t)&(yte==1)).sum()+C_fp*((p>=t)&(yte==0)).sum()) for t in ths]
best=int(np.argmin(costs))
t=ths[best]
print(f"cost-optimal threshold: {t:.4f}  -> flags {(p>=t).mean():.2%} of txns, catches
{((p>=t)&(yte==1)).sum()/yte.sum():.0%} of fraud, cost ${min(costs):,}")
print(f"vs threshold 0.5:       flags {(p>=0.5).mean():.2%}, catches
{((p>=0.5)&(yte==1)).sum()/yte.sum():.0%}, cost ${C_fn*((p<0.5)&(yte==1)).sum()
+C_fp*((p>=0.5)&(yte==0)).sum():,}")
```

Listing 7 — Feed ranking: value model plus MMR diversity re-ranking

CTR-only ranking produces a clickbait monoculture with negative value; a multi-head value model fixes the objective; MMR trades a little value for topic coverage.

```
"""Listing 7: feed ranking — a multi-objective value model plus diversity re-ranking
(MMR).
Production feeds rank by a weighted value model over several predicted engagement heads
(click/like/share/hide), not one CTR head. Then a re-ranker trades a little value for
diversity —
greedy top-K by value alone floods the feed with near-duplicate items from the dominant
topic.
MMR: pick argmax [ lambda*value - (1-lambda)*max_sim_to_already_picked ]."""
import numpy as np
rng=np.random.default_rng(0)
ni,dt,K=500,16,20
topic=rng.integers(0,8,ni); temb=rng.normal(0,1,(8,dt)); temb/
=np.linalg.norm(temb,axis=1,keepdims=True)
emb=temb[topic]+rng.normal(0,0.25,(ni,dt)); emb/
=np.linalg.norm(emb,axis=1,keepdims=True)
# engagement heads correlate with a dominant clickbait topic
pclick=np.clip(0.05+0.15*(topic==0)+rng.normal(0,0.02,ni),0,1)  # topic 0 = clickbait
plike =np.clip(0.02+0.03*(topic==1)+0.01*rng.random(ni),0,1)     # topic 1 = crowd-
pleaser
pshare=np.clip(0.005+0.02*(topic==1)*rng.random(ni)+0.005*rng.random(ni),0,1)
```

```
phide =np.clip(0.002+0.03*(topic==0)*rng.random(ni),0,1)         # clickbait gets
hidden more
value=1.0*pclick+3.0*plike+8.0*pshare-20.0*phide            # business-tuned head
weights
ctr_only=np.argsort(-pclick)[:K]
val_only=np.argsort(-value)[:K]
def mmr(lmb):
    picked=[int(np.argmax(value))]
    while len(picked)<K:
        rest=[i for i in range(ni) if i not in picked]
        sim=emb[rest]@emb[picked].T
        sc=lmb*value[rest]-(1-lmb)*sim.max(1)
        picked.append(rest[int(np.argmax(sc))])
    return np.array(picked)
def report(name,idx):
    print(f"{name:18s} value={value[idx].sum():6.2f}  topics={len(set(topic[idx]))}  "
          f"top-topic share={np.bincount(topic[idx]).max()/K:.0%}
hides={phide[idx].sum():.2f}")
report("CTR-only",ctr_only)
report("value model",val_only)
for l in [0.9,0.7,0.5]: report(f"MMR lambda={l}",mmr(l))
```

Pitfalls, comparisons and practical tips

The classic design-interview mistakes, in the order they usually happen: jumping to a model before framing the problem (always frame first — label, metrics, constraints, baseline); proposing a single-stage architecture for a web-scale corpus (the latency arithmetic of Listing 1 forbids it); optimizing a proxy without guardrails (Listing 7's clickbait feed); trusting offline metrics computed on logged clicks (Listing 5); reporting accuracy or ROC-AUC on heavily imbalanced problems (Listing 6); forgetting that ads and risk systems consume the probability itself, so calibration is launch-blocking (Listing 3); designing only the model and not the data path (feature store, training/serving consistency, label pipelines with delay and censorship); and never mentioning cold start, exploration, or feedback loops — the dynamics that appear only after the system ships.

Stage-by-stage comparison — what each funnel stage is for:

	Candidate generation	Ranking	Re-ranking
Input size	10^6–10^9	10^2–10^4	10^1–10^2
Model	Two-tower + ANN, heuristics	Deep model, cross-features	Rules, MMR, policy
Optimized for	Recall	Precision at top	List-level goals
Features	Separable (user ⊗ item)	User × item interactions	Whole-list context
Failure cost	Unrecoverable miss	Wrong order	Poor variety / policy breach
Typical metric	Recall@k	NDCG, AUC, calibration	Diversity, guardrails

Metric confusion table — which number to reach for:

Situation	Use	Avoid	Why
Retrieval stage	Recall@k	Precision	Misses are unrecoverable downstream
Ranked list quality	NDCG@k, MRR	Accuracy	Position matters
Heavy class imbalance	PR-AUC, recall@fixed-FPR	Accuracy, ROC-AUC	Negatives swamp both (Listing 6)
Probabilities consumed downstream	Calibration (ECE, reliability)	AUC alone	Monotone metrics are blind to scale (Listing 3)
Comparing two rankers cheaply	Interleaving	Underpowered A/B	10–100× sensitivity per impression
Launch decision	A/B on product + guardrails	Offline-only wins	Logged-data bias (Listing 5)

Offline-to-online checklist before shipping a ranking change: (1) offline gain measured with propensity-corrected or randomized data, not raw clicks; (2) power analysis says the A/B can detect the expected lift; (3) feature parity verified between training pipeline and serving path (log the served features and diff); (4) shadow deployment shows sane score distributions and latency; (5) canary on a small slice with guardrails wired to auto-rollback; (6) experiment runs past novelty effects with pre-registered stopping.

Cold-start decision guide: new item + content features available → content-to-latent bridge or content-aware item tower (Listing 4); new item, no features → popularity prior plus forced exploration share; new user → onboarding signals, contextual features (geo, device, time), population priors; both new → contextual bandit territory; and in every case, an exploration budget is what converts "cold" into "warm" — a purely greedy system stays ignorant forever.

Latency budget rules of thumb (order-of-magnitude, worth saying aloud): ANN retrieval over 10^8 vectors: single-digit ms (Listing 2 scale-down measured sub-ms at 200k); feature-store point lookups: ~1 ms; scoring ~500 candidates with a distilled ranker: 5–10 ms batched; whole ranking slice of a 200 ms page budget: 20–50 ms. If a proposed component cannot fit, the answer is precomputation (embeddings, aggregates), distillation, caching, or moving the work offline — not hoping.

Freshness ladder — how quickly each part of the system learns: re-ranking rules and value-model head weights: instant (config); online feature aggregates: seconds–minutes (streaming); ranker retrain: hours–daily; two-tower retrain + full re-index: daily–weekly (they must ship together — the index and towers share a geometry); label-delayed learners (fraud chargebacks, conversions): bounded by label maturity, mitigated with

provisional labels and backfill. Match the interviewer's freshness question to the right rung rather than promising "real-time everything."

Interview questions and answers

Q1. How do you start an ML system design question?

Frame before designing: clarify the business goal and whether ML is warranted (propose the heuristic baseline first), choose the prediction target/label and name its biases, build the metric hierarchy (offline model metrics → online product metrics → north-star business metric, plus guardrails), and state the scale/latency/freshness constraints — because the constraints select the architecture. Interviewers grade the framing as heavily as the architecture; diving straight into model choice is the classic junior failure.

Q2. Why is the choice of label the highest-leverage decision?

The system optimizes exactly what is labeled, not what was intended. Clicks are abundant and immediate but reward clickbait (Listing 7: a pCTR-only feed had negative total value); purchases align with value but are sparse and delayed; fraud labels arrive weeks late and are censored (blocked transactions never get labels). Every downstream problem — position bias, feedback loops, misaligned incentives — enters through the label.

Q3. Describe the candidate generation → ranking → re-ranking funnel and why it exists.

Retrieval reduces 10^6–10^9 items to hundreds–thousands with cheap separable models (ANN over embeddings, heuristic sources), the ranker scores those with an expensive cross-feature model, and the re-ranker applies list-level logic (diversity, freshness, policy). It exists because of arithmetic: the good model cannot afford the corpus and the affordable model cannot rank. Listing 1: expensive-on-random-subset 0.01, cheap-on-everything 0.03, funnel 0.37 recall@10.

Q4. Why is retrieval optimized for recall and ranking for precision?

A retrieval miss is unrecoverable — no later stage sees the dropped item — so the first stage's only job is not to lose good candidates; false positives it lets through cost only ranking compute. The ranker then supplies precision at the top, where user attention lives. The candidate-pool size is the dial between them (Listing 1: recall@10 climbs 0.10 → 0.48 as the pool grows 50 → 5,000).

Q5. Explain the two-tower architecture and the constraint that makes it fast.

Separate encoders map the user and the item to vectors scored by a dot product, trained contrastively on interactions. Because the item side depends only on the item, all item vectors are precomputed into an ANN index; a request costs one user-tower pass plus one nearest-neighbor query. The price of that separability: no user×item cross-features — which is exactly what the ranking stage adds back.

Q6. Why not brute-force nearest-neighbor search, and what are the main ANN families?

Exact search is O(N·d) per query — milliseconds at 10^5 items, seconds at 10^8 (Listing 2: 3.7 ms at N=200k). IVF clusters the corpus and scans only `nprobe` cells (measured: recall 0.22 at 0.4% scanned up to 0.94 at 25%); HNSW greedily searches a layered proximity graph (great recall/latency, more memory, incremental inserts); product quantization compresses vectors into codebook codes for 10–50× memory savings and composes with IVF for billion-scale corpora.

Q7. What operational issues come with an ANN index?

The index is built offline, so new items need an insertion path (HNSW inserts incrementally; IVF assigns to existing cells, degrading if the distribution drifts); retraining the towers changes the embedding geometry, invalidating the whole index, so tower retrains and full re-indexes are scheduled together; and recall/latency must be re-tuned per corpus since the nprobe/ef dials shift with data.

Q8. What is the cold-start problem and how severe is it really?

Interaction-learned representations are exactly useless for entities with no interactions — not degraded, random. Listing 4: MF recall@10 was 0.537 on warm items and 0.055 on cold items versus 0.050 for random guessing, because a cold item's embedding never received a gradient.

Q9. Name the standard cold-start mitigations.

Content bridges (map item features into the interaction-embedding space using warm items as supervision — Listing 4: 0.055 → 0.602); popularity/heuristic priors and onboarding questionnaires for new users; exploration slots (ε-greedy or optimism bonuses) so cold items earn interactions; and hybrid towers that consume content features plus a learned ID embedding, so a new item is usable at upload and improves as data accrues.

Q10. Why do offline and online evaluation disagree for ranking systems?

The logs were generated by the old system: you observe clicks only on what it showed, at the positions it chose, with position bias entangled in every label. Predicting logged clicks therefore rewards imitating the old ranker. Listing 5: naive and IPS models tie on offline AUC (0.909 vs 0.908, naive slightly ahead) while the online metric separates them (true-best-at-rank-1: 0.282 vs 0.343).

Q11. What is position bias and how is it corrected?

Items shown higher get clicked more independent of relevance, so click-trained models learn "was ranked high" as a feature of relevance — self-reinforcing, since the model then decides what ranks high. Corrections: inverse propensity scoring (weight each example by 1/P(exposure)), click models that estimate propensities jointly with relevance, and injecting randomization into logging (small shuffles of top positions) to make propensities estimable.

Q12. Sketch the A/B testing methodology and its statistical cost.

Randomize users into arms, serve each arm its system, compare online metrics with a significance test; validate the pipeline with an A/A test, pre-register the sample size, watch guardrails, and run long enough to wash out novelty effects. The cost is traffic: detecting a +2% relative lift on a 2% CTR needs roughly a million users per arm just for 50% power (Listing 5) — which is why variance reduction (CUPED) and interleaving exist.

Q13. What is interleaving and when is it preferred?

Merge two rankers' results into one list per query and attribute each click to the ranker that contributed the item; the paired within-query comparison removes between-user variance, making it 10–100× more sensitive per impression than A/B. Preferred for fast ranker-vs-ranker iteration; it only answers "which is preferred," so a final A/B still measures business-metric deltas before launch.

Q14. At a 0.2% fraud rate, why are accuracy and ROC-AUC misleading, and what do you use?

"Always legit" scores 99.8% accuracy, and ROC-AUC conditions on the negative class — with 99.8% negatives, ranking most negatives below positives is easy credit (Listing 6: ROC-AUC 0.982). Precision-recall conditions on the rare positives and exposes the truth (PR-AUC 0.425; at 90% recall, precision was 0.04). Use PR curves plus recall-at-fixed-precision or precision-at-fixed-review-budget.

Q15. How do you set the operating threshold for a fraud model?

From the cost matrix, not by default: with a missed fraud at $500 and a manual review at $5, sweeping thresholds for minimum expected cost gave a threshold flagging 2.8% of transactions, catching 88% of fraud at $36.6k, versus 59.6k at threshold 0.5 (Listing 6). Production systems make it three-way — approve / manual-review queue / block — with two thresholds set by review capacity and risk appetite.

Q16. What makes fraud labels hard?

They are delayed (chargebacks land weeks later, so recent data is unlabeled or provisionally negative) and censored (blocked transactions never receive a label, so the training distribution is only what past systems allowed — selective labels bias). Mitigations: label-maturity windows, treating review outcomes as labels, and propensity-aware training on the allowed subset.

Q17. Why do ads systems need calibrated probabilities when ranking metrics look fine?

The auction ranks by expected value pCTR × bid and prices from it, so the probability is consumed as a number, not an ordering. Listing 3: downsampled training inflated mean pCTR to 0.141 vs a true 0.018 with AUC identical (0.919) before and after correction — yet the miscalibrated ranking realized 919 units of value versus 1,095 calibrated, because miscalibration interacts with heterogeneous bids to reorder the auction.

Q18. Give the downsampling calibration correction and when it applies.

If negatives are kept with rate s, a model calibrated on the downsampled data with prediction p is corrected by $q = p / (p + (1 - p)/s)$, the exact Bayes adjustment for the changed prior. It applies whenever you subsample one class for training efficiency or balance and still need real-world probabilities (ads, fraud, risk) — Listing 3 measured ECE 0.123 → 0.0007.

Q19. Why do feeds rank by a multi-head value model instead of one engagement model?

Single-signal optimization Goodharts: pCTR-only produced a 100%-clickbait feed with high hides and negative total value (Listing 7). Separate heads (click, like, share, hide, report) combined by business-tuned weights let the product encode what it values (e.g. 1·click + 3·like + 8·share − 20·hide) and retune the trade-off without retraining the heads. The weights are a product decision set by leadership plus experimentation — say that explicitly.

Q20. What is MMR and what does diversity re-ranking cost?

Maximal Marginal Relevance greedily picks argmax λ·value − (1−λ)·max-similarity-to-already-picked, directly trading item value against redundancy. Listing 7: λ=0.9 kept 99.6% of value while breaking the single-topic monoculture; λ=0.5 covered all 8 topics (top-topic share 100% → 35%) for a 33% value haircut. The λ dial is tuned by A/B against long-term retention, which diversity usually helps.

Q21. How do fraud detection and spam filtering differ in their cost asymmetry?

They are mirror images: in fraud, the expensive error is the false negative (missed fraud costs hundreds of dollars; a false positive costs a $5 review), pushing the threshold toward high recall; in spam, the catastrophic error is the false positive (legitimate mail lost), so the operating point pins FPR very low (e.g. ≤0.1%) and the metric is recall at that pinned FPR. Same math, opposite corner of the PR curve.

Q22. What is a feature store and why do real-time ML systems need one?

A shared system that computes, stores, and serves feature values with two synchronized paths: batch/streaming pipelines that maintain aggregates (a card's transactions-per-hour, a user's 30-day CTR) and a low-latency online store for serving. It exists to meet tens-of-ms budgets (you cannot scan history at request time) and to guarantee training/serving consistency — the same feature definition and time semantics in both — killing a major class of training–serving skew and label leakage (point-in-time correctness).

Q23. What are feedback loops in deployed rankers and how do you counter them?

The model chooses what is shown, which generates the next training data: popular items accumulate impressions and thus labels, cold and niche items never get the chance, and the model's own biases are laundered into "ground truth." Counters: exploration traffic, propensity logging and IPS training, holdout slices served by a simple policy for unbiased measurement, and monitoring diversity/concentration as first-class metrics.

Q24. Lexical vs semantic retrieval in search — why hybrid?

BM25/inverted-index retrieval matches exact terms — unbeatable on rare entities, model numbers, and codes but blind to synonyms; dense two-tower retrieval matches meaning but can miss must-match tokens and fails on out-of-vocabulary strings. Their failure sets barely overlap, so production search retrieves with both and fuses scores (e.g. reciprocal rank fusion or a learned combiner) before ranking.

Q25. What monitoring does a deployed ranking/decision system need?

Four layers: system health (latency, error rates, funnel-stage timeouts); prediction health (score distributions, calibration drift, feature null-rates against training profiles); data drift (input distribution shifts, upstream schema changes); and outcome health (online metrics, guardrails, delayed-label backfill accuracy). Plus retraining triggers — scheduled, drift-triggered, or continuous — with shadow deployment and canary before full rollout.

Q26. When should you NOT use ML for a design problem?

When a rule captures most of the value, when there is no measurable objective or no feedback data to learn from, when the cost of errors is unacceptable without human review anyway, or when the latency/complexity budget cannot fit inference. The strong-candidate move is proposing the heuristic baseline first regardless: it de-risks the launch, provides the comparison bar, and often survives beside the model (rules-beside-model in fraud and spam).

Q27. An offline AUC improvement failed to move the online metric in an A/B. Diagnose.

Ordered hypotheses: the offline metric was computed on biased logs so the "improvement" was imitating the old ranker (Listing 5's trap); the test was underpowered for the true effect size (Listing 5's power table); training–serving skew — features computed differently online; the improvement lives below the funnel stage that dominates errors (better ranking cannot fix retrieval misses); or the online metric saturates (AUC gains in a score region that never changes the served top-k). Each has a distinct check: counterfactual/randomized evaluation, power analysis, feature-parity logging, per-stage recall audits, and rank-change-rate measurement.

Q28. How would you serve a model under a 20 ms end-to-end ranking budget?

Spend the budget where it buys quality: ANN retrieval a few ms (Listing 2's sub-ms IVF queries), precomputed features from the online feature store (no request-time aggregation), a distilled/quantized ranker sized so batch-scoring ~500 candidates fits the remaining ms, caching for repeated (user, context) requests, and strict per-stage timeouts with graceful degradation — fall back to the previous stage's ordering rather than failing the page.